

The comonadic harness — how it works

The closed alphabet, the coalgebra, and the tool-call cycle

This is a literate module. The prose you are reading is not seen by the compiler; the indented > lines are real Haskell, compiled as part of the library, so every example below is guaranteed to build against the current API. Reference documentation for each type and function lives in its own module's Haddock; this document is the guided walk through how the pieces fit.

```
module Harness.Tutorial
( demoStart
, demoTree
, demoHypo
, demoTrace
, demoRisk
, demoGateEqualsRoot
) where

import Control.Comonad (extract)
import Control.Comonad.Cofree (Cofree)
import Data.List (isInfixOf)
import Harness.Alphabet
import Harness.State (S (...), Ctx, Mode (Working))
import Harness.Coalgebra (harness)
import Harness.Interp (Ev)
import Harness.Probe (Hypo (...), Risk, assess, governed, probe)
```

The closed alphabet

Everything the harness can do is one of three constructors:

```
data HarnessF x
  = Render Request (Either Refusal Response -> x)  -- ask the model
  | Perform Call (Obs -> x)                          -- run a tool
  | Halt Outcome                                     -- stop
```

Render asks the oracle: it carries the **Request** the model sees and a continuation that consumes whatever comes back — a **Refusal** or a **Response**. **Perform** runs a tool **Call** against the world and continues with the **Obs** it produces. **Halt** stops with an **Outcome**.

The *positions* (**Request**, **Call**, **Outcome**) are what the harness does and are fully determined; the *directions* (the ... -> x functions) are what it does not control. All nondeterminism lives in the directions. The alphabet is closed: adding a fourth constructor without a matching case in the interpreter makes `-Wincomplete-patterns` fail the build, so there is deliberately no wildcard to absorb one.

The coalgebra

The control flow of the whole agent is one pure function:

```
step :: S -> HarnessF S
```

Given a state it returns the single next action and, in the action's direction, how each possible reply continues. It is deterministic — it always knows what it does next; what it cannot know is what comes back, which is why the successor is a function of the reply rather than a plain state.

Unfolding the tree

`harness` unfolds the coalgebra from a start state into the lazily-grown tree of every reachable future, annotating each node with a `Ctx` (what analysis reads):

```
harness :: S -> Cofree HarnessF Ctx
harness = unfold (\s -> (view s, step s))
```

Our first compiling values — a start state, and the tree it denotes:

```
-- | An empty starting state: no transcript, no pending calls, a 500-token
-- budget, in 'Working' mode.
demoStart :: S
demoStart = S { transcript = [], pending = [], budget = 500, mode = Working }

-- | The tree of every reachable future from 'demoStart' - a denotation.
demoTree :: Cofree HarnessF Ctx
demoTree = harness demoStart
```

`demoTree` is a *denotation*, not a runtime structure: nothing has run, and the implementation never serialises or caches it. It is a value you can walk.

One interpreter, two consumers

A single interpreter walks that shape. The real driver, `run`, walks it in IO against a live model and a real world. The analyser, `probe`, walks the same shape *purely*, to a bounded depth, collecting a trace. Because both factor through the one interpreter, what a real run does and what a dry-run predicts agree by construction — there is no second hand-written loop to drift.

The tool-call cycle

This is the round trip a tool call actually makes. Walk it once:

1. `step` reaches a `Render`, carrying the `Request` — the projected prompt plus the tools currently afforded.
2. The oracle answers with a `Response` whose `calls` are the tools the model asked for (from a live model these are decoded from its `tool_calls`).
3. Those calls become the state's `pending` queue.
4. The next `step` admits them: only *afforded* calls become a `Perform`; an unafforded or hallucinated one is repaired into a synthetic error `Obs` instead, never crashing.
5. `Perform` runs one call against the world; the resulting `Obs` is recorded into the transcript, so the model sees it on the next `Render`. Multi-call responses drain through `pending` one `Perform` at a time.

To watch it without a live model, stand in a pure `Hypo` — a model of the oracle and world. This one asks for a `read` on the first turn, then, seeing the read in the prompt, declares itself done (a `Response` with no calls, which halts):

```

-- | A pure stand-in oracle\world: ask for one @read@, then declare done.
demoHypo :: Hypo
demoHypo = Hypo
  { guessOracle = \ (Request (Prompt prompt) tools) ->
    if null tools
    then Right (Response "summarised" [] (Usage 40 10))
    else if "read" `isInfixOf` prompt
    then Right (Response "done, nothing more to do" [] (Usage 40 10))
    else Right (Response "reading the file"
      [Call "read" "{\\"path\\":\\"README.md\\"}"]
      (Usage 120 30))
    , guessWorld = \c -> Obs (tool c ++ " -> ok")
  }

```

probe drives the tree under that hypothesis to a fuel bound and returns the event trace:

```

-- | The event trace of driving 'demoTree' under 'demoHypo' to fuel 8.
demoTrace :: [Ev]
demoTrace = probe demoHypo 8 demoTree

```

Evaluating demoTrace gives:

```

[ Asked Working
, Did (Call {tool = "read", args = "{\\"path\\":\\"README.md\\"}"}))
, Asked Working
, Ended (Done "done, nothing more to do")
]

```

Read left to right, that is the cycle: Asked (a Render went out), Did (a Perform ran the read), Asked again (the next Render, now with the read in the prompt), then Ended (the coalgebra Halted with Done). No tool effect happened — guessWorld stubbed the read — but the *shape* of the exchange is exactly what a live run walks.

The provider seam

Execution pairs the tree against an environment, **Env**, with exactly two fields: an **oracle** (the model) and a **world** (the tool executor). The LLM lives in one field and nowhere else. In the shipped demo the live world is a sandboxed executor that really runs **read/write/bash/commit**; the scripted world is a stub. Swapping either is a local change; the coalgebra never mentions them.

Analysis: the gate

Because the tree branches on things you do not control, you cannot fold it — but under a Hypo it collapses to a path you can. **assess** scores one node's future:

```

-- | The forecast at 'demoStart': four steps ahead, terminates, no writes.
demoRisk :: Risk
demoRisk = assess demoHypo 8 demoTree

```

which for demoStart is:

```

Risk { stepsAhead = Sum {getSum = 4}
    , terminates = Any {getAny = True}
    , irreversible = []
    , compactions = Sum {getSum = 0} }

```

That reads as: four steps to the horizon, it terminates, it touches no irreversible tools, and it triggers no compactions. `governed h n = extend (assess h n)` lifts that one-node score to *every* reachable node, and `extract` reads the score at the current node — a pre-action gate. `extract (governed demoHypo 8 demoTree)` equals `demoRisk` above:

```
-- | The gate at the root equals 'demoRisk' - @extract . governed@ recovers @assess@.  
demoGateEqualsRoot :: Bool  
demoGateEqualsRoot = extract (governed demoHypo 8 demoTree) == demoRisk
```

The compaction law lives here too: `respectsBehaviour` probes a compacted and an uncompact state under the same `Hypo` and returns both behavioural shadows, which are equal exactly when compaction did not change what the agent does. Why that is the right property — and why the answer is a measured rate, not a proof — is the subject of the companion essay, *Why an agentic harness is a comonad* ([docs/motivation.pdf](#)).